Ronak Katariya
D20A / 29

Experiment : 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE.

Theory:

1. Introduction to Smart Contracts

A **Smart Contract** is a self-executing digital program stored on a blockchain that automatically enforces rules and agreements when predefined conditions are met. These contracts eliminate the need for intermediaries, ensuring a secure and transparent system.

Smart contracts are commonly written in Solidity and executed on the Ethereum Virtual Machine, which is the runtime environment for blockchain-based applications.

2. Ethereum Blockchain Overview

The Ethereum blockchain is a decentralized platform that supports smart contracts and decentralized applications (DApps). It enables developers to build secure and automated systems without relying on central authorities.

3. Key Concepts

a) Solidity Programming Language

- A high-level, contract-oriented programming language.
- Used to write smart contracts for Ethereum.
- Syntax is similar to JavaScript and C++.

b) Ethereum Virtual Machine (EVM)

- Executes smart contracts on the blockchain.
- Ensures consistent execution across all nodes.
- Maintains the state of contracts.

c) Remix IDE

Remix IDE is a browser-based development tool used to:

- Write Solidity code
- Compile smart contracts
- Deploy and test contracts
- Interact with deployed contracts

4. Significance of Smart Contracts in Ethereum

- **Automation**

Smart contracts automatically execute predefined actions when conditions are met, reducing manual intervention.

- **Transparency**

  All contract code and transaction details are publicly visible on the blockchain.

- **Security**

  Once deployed, smart contracts are immutable and resistant to tampering.

- **Cost-Effectiveness**

  Eliminates intermediaries, reducing operational and transaction costs.

- **Trustless Environment**

  Participants can interact without needing to trust each other, as the code governs execution.

5. Role in Modern Applications

Smart contracts serve as the foundation for:

- **Decentralized Applications (DApps)**
- **Decentralized Finance (DeFi)** systems
- Token creation and management
- Voting and governance systems

6. Performing Transactions using Smart Contracts

Transactions in smart contracts involve:

- **Deployment Transaction** → Deploying the contract to the blockchain
- **State-Changing Transactions** → Functions that modify data (require gas)
- **View Transactions** → Functions that only read data (no gas required)

All transactions are verified and recorded on the blockchain, ensuring reliability and traceability.

<u>Implementation:</u>

<u>Code:</u>

AccessControl.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

ıtract AccessControl {
```

```
    mapping(address => mapping(address => bool))
    private accessList;

  function grantAccess(address doctor) public {

    accessList[msg.sender][doctor] = true;

  }

  function revokeAccess(address doctor) public {

    accessList[msg.sender][doctor] = false;

  }

   function hasAccess(address patient, address doctor)
    public view returns (bool) {

    return accessList[patient][doctor];

  }

}
```

PatientRecord.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

interface IAccessControl {

   function hasAccess(address patient, address doctor)
    external view returns (bool);

}

contract PatientRecord {

  address public accessControlAddress;

  constructor(address _accessControlAddress) {

    accessControlAddress = _accessControlAddress;

}

struct Record {

  string data;

  uint timestamp;

}

mapping(address => Record[]) private records;

function addRecord(string memory _data) public {

        records[msg.sender].push(Record(_data,
block.timestamp));

}

  function getRecords(address patient) public view
returns (Record[] memory) {

        IAccessControl ac =
IAccessControl(accessControlAddress);

  require(

      patient == msg.sender || ac.hasAccess(patient,
msg.sender),

    "Access Denied"

  );

  return records[patient];

}
```

Output:

First got error

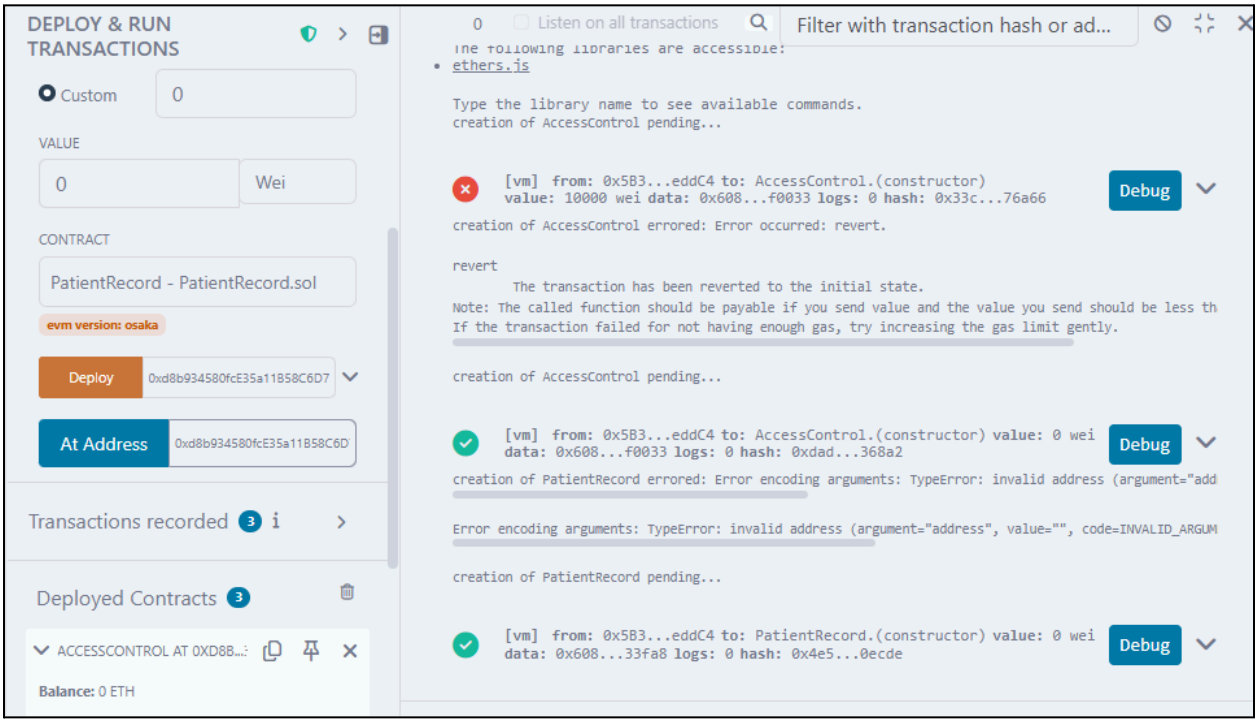
Ronak Katariya
D20A / 29

```
[vm] from: 0x5B3...eddC4 to: AccessControl.(constructor)
value: 10000 wei data: 0x608...f0033 logs: 0 hash: 0x33c...76a66
creation of AccessControl errored: Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Note: The called function should be payable if you send value and the value you send should be less th
If the transaction failed for not having enough gas, try increasing the gas limit gently.

creation of AccessControl pending...
```

Got deployed first one:

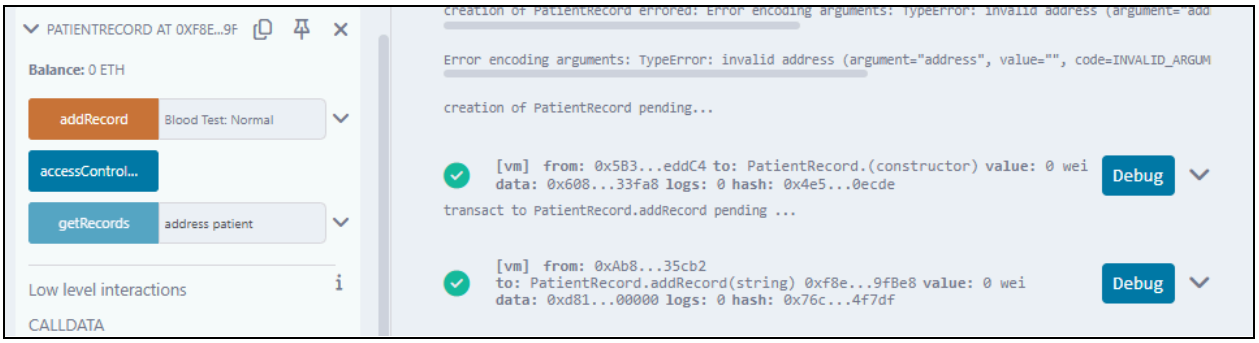
```
[vm] from: 0x5B3...eddC4 to: AccessControl.(constructor) value: 0 wei
data: 0x608...f0033 logs: 0 hash: 0xdad...368a2
creation of PatientRecord errored: Error encoding arguments: TypeError: invalid address (argument="add

Error encoding arguments: TypeError: invalid address (argument="address", value="", code=INVALID_ARGUM

creation of PatientRecord pending...
```

Second one deployed

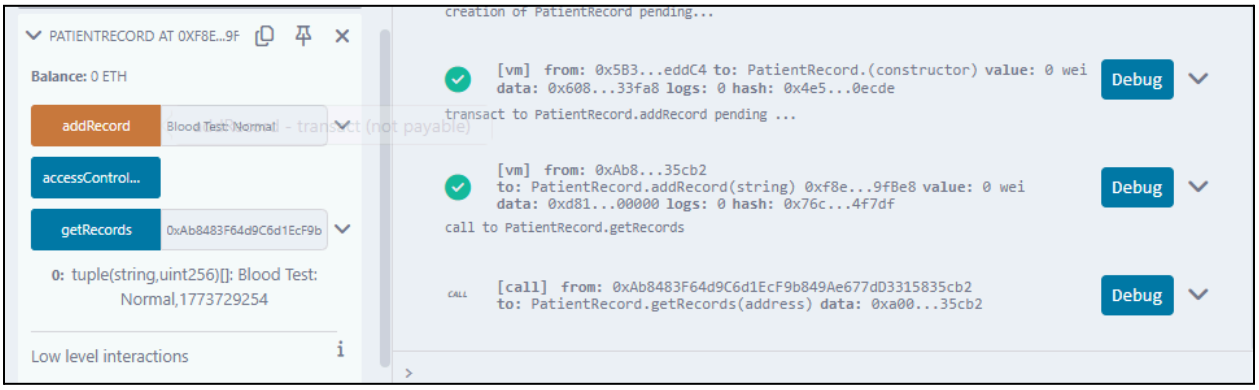


Added the record

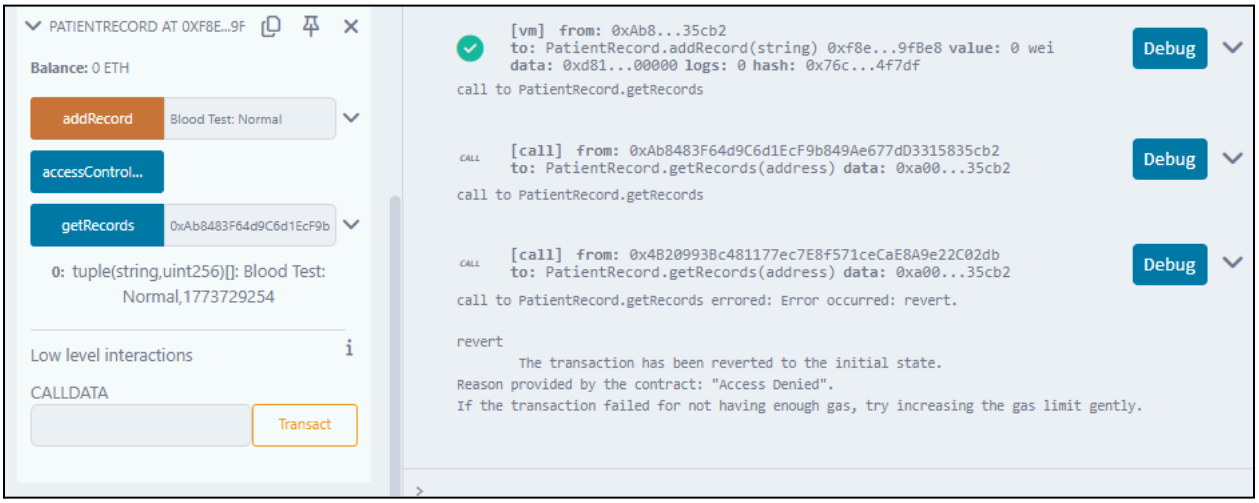
Ronak Katariya
D20A / 29



Get the record

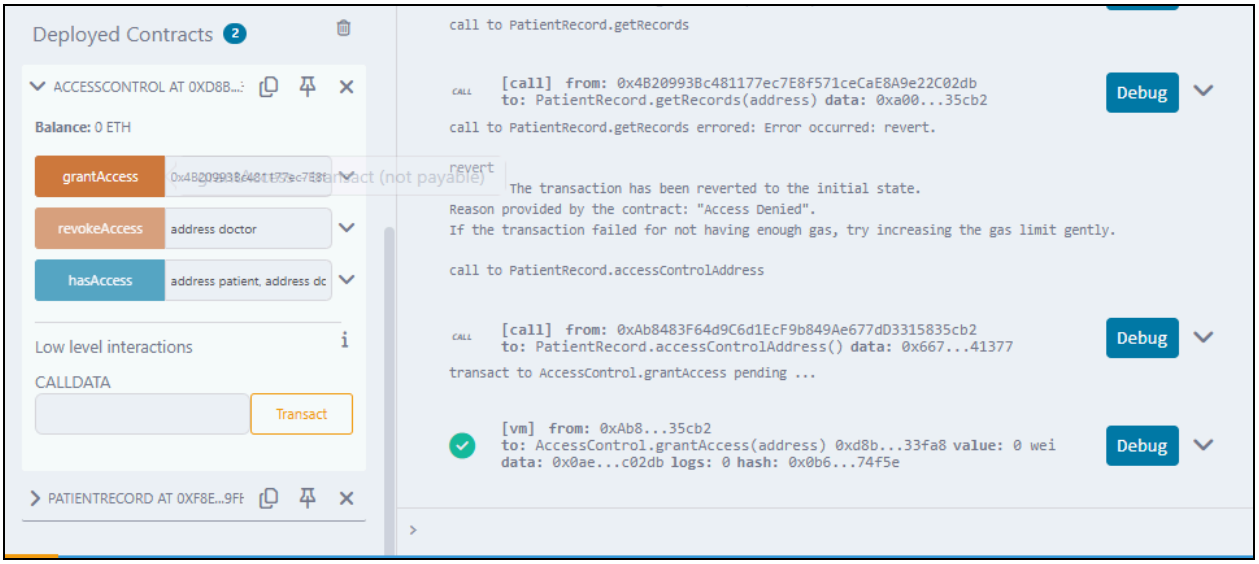


Doctor Tries to Access (FAIL CASE)

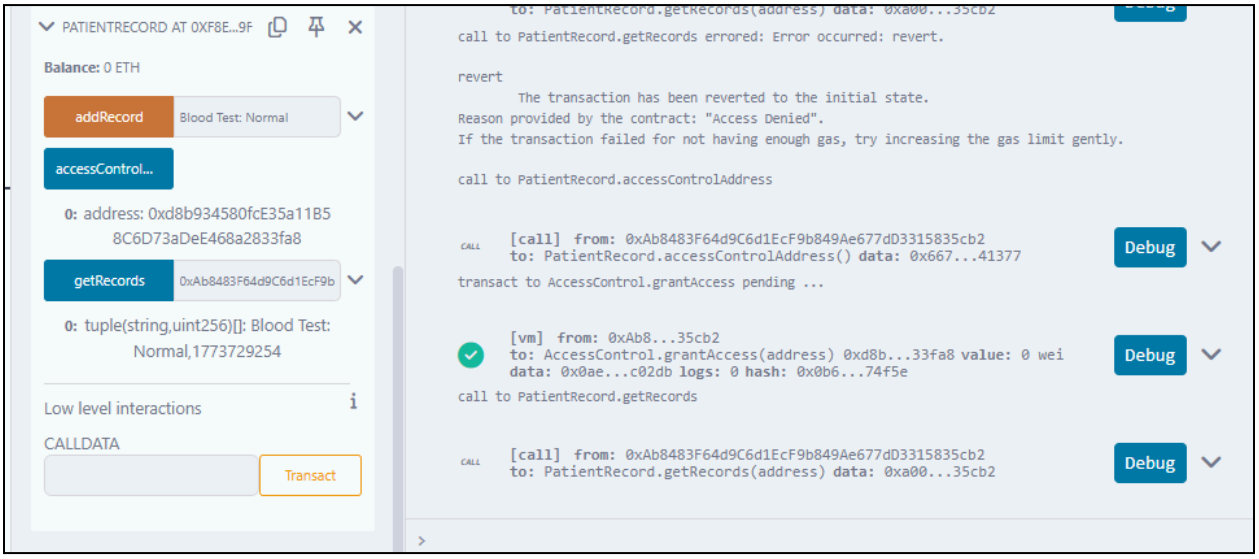


Patient Grants Access

Ronak Katariya
D20A / 29

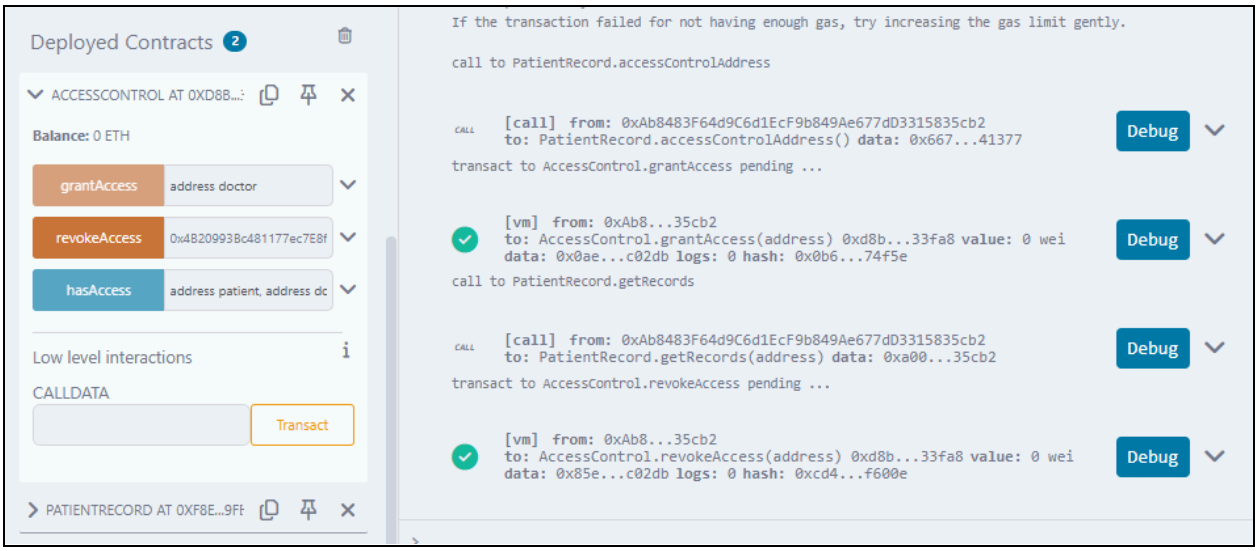


Doctor Access Again (SUCCESS CASE)



Access revoked

Ronak Katariya
D20A / 29



Again the access has been denied

Conclusion:

Smart contracts revolutionize digital agreements by enabling secure, automated, and transparent execution of logic on the blockchain. Using tools like Remix IDE and languages like Solidity, developers can efficiently create, deploy, and interact with blockchain-based applications.